Lru Cache

AlgoExpert · Very Hard · Linked Lists

The problem

- InsertKeyValuePair, getValueFromKey (null if missing) and getMostRecentKey, evicting the least recently used key at capacity.
- Hash map + doubly linked list.

Intuition

- A hash map for lookup plus a doubly linked list for recency order gives constant-time operations.

Brute force vs optimal

- List with linear search → O(n) ops
- Hash map + doubly linked list → O(1) all ops

Approach

- 1. Map keys to nodes
- 2. Move accessed nodes to the head
- 3. Evict the tail at capacity
- 4. Most recent key is the head

Worked example

most → least recent




```
put(b, 2)
```

put b

Worked example

most → least recent




```
put(a, 1)
```

put a

Worked example

most → least recent




```
put(c, 3)
```

put c

Worked example

most → least recent



`get(a)`

get a

Worked example

most → least recent




```
put(d, 4) → evict b
```

put d, evict b

Java solution (1/2)

```java
public class LruCache {
    private static class Node { String key; int value; Node prev, next; Node(String k, int v) { key = k; value = v; } }

    private final int capacity;
    private final Map<String, Node> map = new HashMap<>();
    private final Node head = new Node(null, 0), tail = new Node(null, 0);

    public LruCache(int capacity) {
        this.capacity = Math.max(1, capacity);
        head.next = tail; tail.prev = head;
    }

    public void insertKeyValuePair(String key, int value) {
        Node n = map.get(key);
        if (n != null) { n.value = value; moveToFront(n); return; }
        if (map.size() == capacity) { Node lru = tail.prev; unlink(lru); map.remove(lru.key); }
        n = new Node(key, value);
        map.put(key, n);
        addFront(n);
    }
```

Java solution (2/2)

```
    public Integer getValueFromKey(String key) {
        Node n = map.get(key);
        if (n == null) return null;
        moveToFront(n);
        return n.value;
    }

    public String getMostRecentKey() { return head.next == tail ? null : head.next.key; }

    private void moveToFront(Node n) { unlink(n); addFront(n); }
    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFront(Node n) { n.next = head.next; n.prev = head; head.next.prev = n; head.next = n; }
}
```

Complexity

- Time: O(1) per op
- Space: O(capacity)

Edge cases & pitfalls

- Update existing key moves it to front
- Capacity ≥ 1
- Sentinel head/tail

Interview tips & follow-ups

- Mention LinkedHashMap

Recap

- A hash map for lookup plus a doubly linked list for recency order gives constant-time operations.
- Complexity: O(1) per op